Atenção

- **Este relatório não é uma entrega separada: ele deve ser enviado junto com o .tar da atividade, no mesma atividade no Google Classroom.**
- **O relatório deve ter como nome as iniciais do seu e-mail em minúsculas (ex.: mla@cesar.school → diretório mla). Fora desse padrão, o relatório é descartado automaticamente.**
- **Cópia identificada, de outro(a) aluno(a) ou de qualquer outra fonte, descarta toda a submissão, código e relatório juntos.**

1. Objetivo

O ProcessFlow é um orquestrador de processos em C que recebe tarefas (programas do sistema), cadastra-as por nome e as executa como processos filhos. Suporta execução sequencial, paralela e em pipe entre tarefas, redirecionamento de entrada/saída/append, diretório de trabalho configurável, e execução em background com controle de jobs. Funciona em modo interativo (processflow>) ou lendo uma lista de comandos de um arquivo .pf.

2. Checklist de Requisitos

Requisito do enunciado	Status	Como testar / evidência
README + Makefile	OK	make compila, make clean remove o binário (commit c0d6355).
Não usa system()/popen()/shell externo	OK	grep -rn "system(|popen(" *.c sem ocorrências — só fork/execvp/wait/dup2/pipe.
Execução simples + exit	OK	task listar /bin/ls -l + run listar, diff idêntico contra teste1-saida.txt (Diário #25).
Sequencial + paralelo	OK	run sequential/run parallel, diff exato contra teste1 (Diário #9/#25).
Pipe	OK	run pipe listar contar → 14, diff exato contra teste2 (Diário #10/#25).
Redirecionamento (input/output/append)	OK	output+input encadeados → 15, diff exato contra teste3 (Diário #11/#25).
Background + jobs + wait	OK	start/jobs/wait com sleep 3, ciclo completo (Diário #12; sem teste oficial cobrindo isso).
workflowFile + workdir	OK	workdir /tmp + ./processflow arquivo.pf, diff exato contra teste3 (decisões #4/#5/#6, Diário #14-19/#25).
Erros + outras situações	OK	argv[0]/argv[1]/argv[2]==NULL checados em todo o dispatch — testado com Enter vazio, comandos sozinhos, arquivo faltando (Diário #13).

3. Como Reproduzir

Compilar:

make

Limpar binário:

make clean

Executar (modo interativo):

```
./processflow
task listar /bin/ls
task contar /usr/bin/wc -l
run listar
run sequential listar contar
run parallel listar contar
run pipe listar contar
output listar saida.txt
```

```
run listar
input contar saida.txt
run contar
task esperar /bin/sleep 3
start esperar
jobs
wait 1
jobs
workdir /tmp
exit
```

Modo workflow: ./processflow teste.pf (roda os comandos do arquivo, ecoando cada linha, sem prompt).

Modo "interativo redirecionado" (como o corretor do professor testa): ./processflow < arquivo.txt — sem prompt, sem eco, só a saída real dos comandos.

Casos de erro pra testar: Enter vazio, task sozinho, wait sozinho, input contar sem arquivo, ./processflow a.pf b.pf (argumentos demais), ./processflow naoexiste.pf — nenhum deve travar o programa.

4. Arquitetura (resumida)

Arquivos e responsabilidades:

1. **main.c → loop interativo/workflow + despacho de task, input, output, append, start, jobs, wait, workdir e run. Toda entrada validada (argv[0]==NULL logo após o parse(), mais argv[1]/argv[2]==NULL em cada bloco que precisa desses argumentos) antes de qualquer strcmp/encontrar_task/atoi — nenhum comando malformado derruba o programa.**
2. **parser.c/parser.h → tokenização de linha em argv[]. strtok usa " \n" como delimitador.**
3. **task.c/task.h → cadastrar_task/encontrar_task, criar_input/criar_output/criar_append. Concluído, testado, commitado.**
4. **job.c/job.h → struct job (jobID, pid, status), iniciar_job, listar_jobs, wait_job. Concluído, testado, commitado.**
5. **exec.c/exec.h → executar(), executar_sequencial(), executar_paralelo(), executar_pipe(), executar_background(). Concluído, testado, commitado.**
6. **common.h → MAXARG, MAXFILE, MAXTASKS, MAXJOBS.**
7. **Makefile → alvo processflow (compila os 5 .c) e clean (remove o binário). Concluído, testado, commitado.**
8. **README.md → arquivos usados, como compilar/executar/limpar, SO usado. Concluído, commitado.**

Decisões técnicas:

1. **Separar o projeto em vários arquivos (main.c/parser.c/task.c/job.c/common.h, criados vazios no commit b73e459, 22/08 09:16) → facilitar o rastreio de erros e dividir uma tarefa grande em desafios menores, além de deixar organizado pra escalar → exec.c ficou de fora dessa leva inicial, entrou depois (decisão 2).**
2. **Separar exec de task (exec.c/exec.h criados no commit 4ba185a, 22/08 14:07) → a ideia original era task.c cuidar de tudo, mas misturar ia complicar o rastreio da execução → exec.c virou responsável por toda a execução de processos, task.c nunca chama fork/exec diretamente.**
3. **Array fixo em vez de lista encadeada (tasks e jobs) → apesar de já ter familiaridade com listas encadeadas, sem necessidade de remover tasks/jobs em tempo de execução, lista encadeada adicionava complexidade sem ganho real → ficou mais simples de implementar e de raciocinar sobre.**
4. **workdir via chdir() no processo pai, não nos filhos → fork() já copia o estado do pai, os filhos herdam automaticamente o novo CWD sem lógica extra → chdir() retornando -1 cobre de graça o erro pedido pelo enunciado (diretório inexistente).**

5. **workflowFile unificado com stdin via FILE *entrada → em vez de duplicar a lógica de leitura pros dois modos, entrada começa como stdin e vira fopen(arquivo) quando há argumento → mesmo fgets serve pros dois modos; argc > 2 e fopen falhando viram os dois erros fatais exigidos pelo enunciado.**
6. **Validação contra os testes oficiais revela dois bugs sérios (23/08, à noite) → diff contra os -saida.txt do professor mostrou que a saída esperada é exata, sem nenhuma mensagem de "sucesso" nossa (removidas) → revelou dois bugs que teste manual não pegava: (a) detecção de modo interativo errada (entrada == stdin não distingue terminal de arquivo redirecionado; corrigido com isatty(fileno(stdin))); (b) saída fora de ordem por buffering de stdout em pipe (corrigido com setbuf(stdout, NULL)). Ambos diagnosticados pela IA, não pelo Rafael (ver Seção 7). Os 3 testes oficiais batem via diff depois da correção.**

5. Estratégias e Diário de Desenvolvimento

Isto é o núcleo do relatório.

5.1 Estratégias da semana

Uma linha por abordagem diferente que você tentou. Se você não mudou de abordagem, preencha só S1.

Estratégia	Nome curto	Contexto	Motivo da troca (se houve)
S1	Aprendizado guiado + planejamento (pré-código)	Quarta e quinta (19-20/08), antes do marco zero (commit b73e459, sábado 22/08): aprendizado guiado de fork/exec/pipe/wait/dup2/strtok com a IA e planejamento da arquitetura (decisão #1). Prática em arquivos de treino como dupepipe.c e um parse() de treino (evidência local, fora do repositório do projeto); outros arquivos de treino dessa fase ficaram só na máquina usada no início, sem versionamento, e não migraram pro repositório atual.	—
S2	Divisão em arquivos + sessões pequenas	Projeto dividido em vários arquivos por responsabilidade desde o planejamento (decisão #1); cada sessão de trabalho focada em completar um arquivo/funcionalidade por vez, em linha com essa divisão. Estratégia organizacional usada durante toda a fase de implementação (fork/exec/pipe/dup2 é o único caminho técnico permitido pelo enunciado, não uma escolha de abordagem — por isso não entra como estratégia própria).	—

5.2 Diário de Tentativas

Uma linha por tentativa relevante, marcando a qual estratégia (S1/S2/S3) ela pertence, obviamente precisa ser em ordem cronológica. A coluna "Quando" precisa bater com a ordem cronológica do evidencias.log, é isso que comprova que o fluxo é real. "Hipótese/Causa" só é preenchida quando o resultado foi falha ou parcial.

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
1	S2	Estrutura de	Sucesso —	—	manhã 22/08	b73e459

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
		arquivos vazia (main.c, parser.c/h, task.c/h, job.c/h, common.h, decisão #1)	esqueleto			
2	S2	task.h: struct + assinaturas; globais pra common.h	Sucesso	—	manhã 22/08	b30d550
3	S2	parser.c com strtok	Bug: main() sobrando, parse() aninhada nele por engano (rodou por extensão do GCC). Corrigido removendo o main()	Sobra de template não removida	manhã 22/08	72037ec
4	S2	Loop principal em main.c	Sucesso	—	manhã 22/08	4be895a
5	S2	encontrar_task v1 (array)	Sucesso — reescrita depois (#6)	—	manhã 22/08	27889f9
6	S2	cadastrar_task/ encontrar_task reescritas (static). Testadas com main.c de teste escrito pela IA, descartado depois	Sucesso — validado sem contaminar código final	—	tarde 22/08	e8a5030
7	S2	Dispatch de task em main.c	Sucesso	—	tarde 22/08	8da634d
8	S2	run <task> via fork+execvp+wait pid	Bug: strtok só cortava espaço; "listar\n" ≠ "listar" cadastrado. Corrigido: delimitador virou " \n"	Confirmado pelo Rafael	tarde 22/08	4ba185a
9	S2	Sequencial em exec.c; discussão com a IA sobre não reciclar o padrão pro paralelo	Sem evidência de log — só discussão de design. Corrigido separando fase fork da fase wait	Discussão, não bug em log	tarde 22/08	651f5dd + log linha 381

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
10	S2	run pipe em exec.c	Bug: pipe criado antes de checar se a task existia. Corrigido: filtra tasks válidas primeiro, cria n-1 pipes	Recurso criado antes de validar dados	noite 23/08	6bdfe09
11	S2	input/output/ append (task.c + dup2 em exec.c)	Sucesso de primeira	—	tarde 23/08	f0b58d0
12	S2	start/jobs/wait (job.c novo)	Sucesso, testado com sleep 3	—	noite 23/08	d79aee7
13	S2	Checagens argv[0]/[1]/[2]== NULL em main.c	Sucesso — 4 casos de erro sem crash	—	noite 23/08 (18:45)	print
14	S2	workdir v1 com bugs (msg copiada, chdir() não checado, %s faltando)	Corrigido, testado válido/inexi stente	Copy-paste sem ajustar	tarde 23/08	print
15	S2	workflowFile v1: chave vazia, print() por printf(), entrada não inicializada	3 bugs corrigidos em revisão, antes de compilar	Revisão de código	tarde 23/08	—
16	S2	fgets==NULL → break, lê de entrada	Sucesso — prompt indevido ainda aparecia (corrigido depois)	—	tarde 23/08	print
17	S2	Prompt condicional + eco condicional	Sucesso	—	tarde 23/08	print
18	S2	exit sem \n final — bug: parêntese errado (strcmp(linha,"exi t\n"==0))	Corrigido pra strcmp(linha ,"exit")==0	Erro de digitação	tarde 23/08	—
19	S2	Bateria workflowFile: argc>2, arquivo inexistente, teste_exit.pf	Sucesso nos 3	—	tarde 23/08	prints
20	S2	Makefile criado	Sucesso — recompila	—	tarde 23/08	prints

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
			do zero, clean remove binário			
21	S2	Download testes.tar do Classroom (Drive/Windows/ WSL)	Sucesso — era .zip com extensão errada	Confusão de infra, não bug	noite 23/08	prints
22	S2	Leitura -entrada/-saida.txt: saída esperada sem mensagens de sucesso	Removidas mensagens de sucesso (main/exec/task.c)	Ver decisão #6	noite 23/08	-saida.txt
23	S2	./main arquivo (workflow) vs. professor usa <	Confirmado: professor usa < (sem eco)	Ver decisão #6	noite 23/08	formato -saida.txt
24	S2	./main < entrada — prompt 4x mesmo redirecionado	Bug corrigido: isatty(stdin) && entrada==stdin	Ver decisão #6	noite 23/08	print antes/depois
25	S2	Validação final: diff 3 testes. teste3 fora de ordem	Corrigido com setbuf(stdout,NULL) — 3 testes limpos	Ver decisão #6	noite 23/08	diffs
26	S2	Binário devia se chamar processflow, não main. Rename no Makefile v1 incompleto	Corrigido as 2 ocorrências — testes revalidados	Confundiu alvo com nome do -o	tarde 24/08	prints
27	S2	Testou setbuf comentado direto no terminal, esqueceu de descomentar	Sem impacto — git diff vazio	Teste exploratório não reproduz o bug (buffer de linha é padrão em terminal)	tarde 24/08	git diff
28	S2	Dúvida se evidencias.log da 2ª máquina era o original	Confirmado — mesmo cabeçalho/hostname nas duas	Dúvida legítima, checada	tarde 24/08	head -c 500

Regra prática: se algo deu muito errado (bug que te travou por horas), a linha da tabela resume em 1 frase e aponta para o trecho do evidencias.log com o detalhe.

6. Evidências

A regra continua: toda afirmação relevante precisa de prova. O que muda é como capturar essa prova, para não depender de print manual toda hora.

6.1 Log automático (faz o trabalho pesado por você)

No início da sessão de trabalho, rode:

```
script -a evidencias.log
```

Isso grava automaticamente todos os comandos e saídas do terminal num arquivo, sem você precisar tirar print de cada linha. Comece a sessão com date, whoami e pwd, assim o log já tem timestamp, usuário e diretório sem esforço extra. **O evidencias.log deve ser enviado dentro do .tar**

6.2 Prints

- Erro principal: o momento do bug mais relevante da semana.
- Validação final: teste passando / saída correta.

Prints extras só se algo excepcional aconteceu e o log sozinho não explica, você precisa ter esse feeling, mas posso dar uma ajuda (ex.: comportamento visual, crash com stack trace longo).

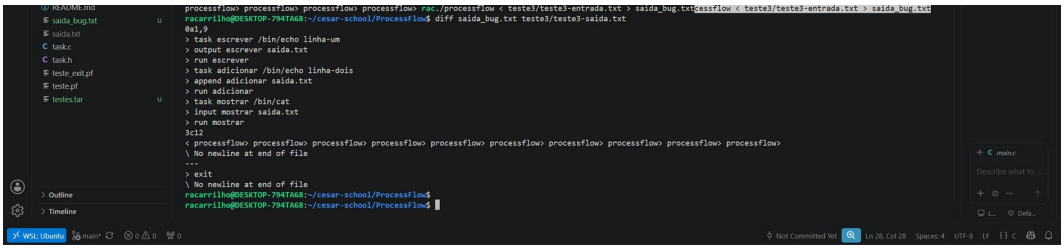
6.1 (preenchido): evidencias.log iniciado em 22/08 09:22:47 (script -a), retomado após troca/reinício de máquina (ver notas no topo do rascunho/documento). Incluído no .tar da entrega.

6.2 (preenchido) — Erro principal:

Recriação controlada dos bugs de isatty (prompt duplicado) e buffering de stdout (saída fora de ordem), comentando temporariamente as correções já commitadas (decisão #6) e revertendo em seguida.



Print 1 — bug do isatty recriado: prompt "processflow>" repetido 4x, grudado na linha de comando seguinte.

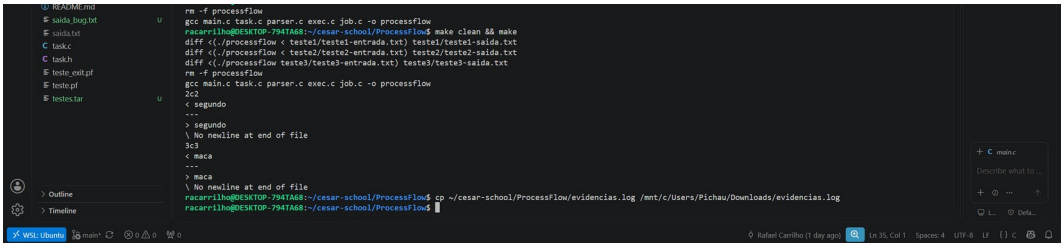


Print 2 — bug do buffering recriado: diff mostrando saída fora de ordem (linha 3 com prompts repetidos em vez de "exit", faltam as 9 linhas de eco esperadas).

Nota de transparência (24/08): os bugs originais foram descobertos e corrigidos em 23/08 (Diário #24/#25); os prints acima são uma recriação de 24/08 pra documentação visual, não a captura original. A recriação ficou registrada no evidencias.log real: "processflow>" repetido 4x na linha 1438 (bug do isatty), diff mostrando "0a1,9"/"3c12" nas linhas 1450/1460 (bug do buffering) — reversão e revalidação limpa dos 3 testes oficiais nas linhas seguintes.

6.2 (preenchido) — Validação final:

Os 3 testes oficiais do professor batendo via diff (Diário #25; teste3 revalidado em modo workflow — argumento, não redirecionamento; teste1/teste2 rodam com <) — validação externa, não auto-declarada.



Terminal: make clean && make, seguido dos 3 diffs oficiais (teste1/teste2/teste3) — únicas diferenças são quebra de linha final ausente (cosmético), sem divergência de conteúdo.

Commits/push relevantes:

run pipe: commit 6bdfe09 (Diário #10). workdir + workflowFile + teste.pf/teste_exit.pf: commit 7722db7. Makefile + README.md: commit c0d6355. Correções de isatty/setbuf + os 6 arquivos de teste oficiais: commit e596dad. Rename main → processflow: commit cd577e0 (confirmado via git pull numa segunda máquina, "Updating e596dad..cd577e0").

7. Uso de IA

Obrigatório preencher mesmo se a resposta for "não utilizei".

Onde usei IA (2–3 linhas):
Usei a IA principalmente como professor/ajudante: ensinou as funções que eu não tinha (fork/exec/pipe/dup2/wait/strtok/isatty/buffering de stdio) com a instrução expressa de não me dar código pronto. Também usei pra revisar e apontar erros no meu próprio código, que foi 100% escrito à mão — única exceção: pedi pra IA escrever as mensagens de erro do programa (ver prompts abaixo). As decisões de arquitetura eu tomei sozinho ou defendi minha posição num debate técnico real com a IA (ver decisão #2 e nota da Seção 4).

Prompts principais (liste, sem história):
(A maioria abaixo é citação literal do que foi digitado; a última é reconstrução próxima do original, ancorada num bug já documentado no Diário.)

- "Tu pode me dar um curso sobre todas as funções que vou ter que aprender?"
- "Preciso que tu me ajude a criar um cronograma pra terminar esse projeto seguindo as instruções no arquivo — reforço que você não deve me dar código ou passar a lógica pra mim, só me guiar enquanto eu tento."
- "O que diabos é strtok?? Me ensina esse troço aí."
- "Ei, essa parte de input/output/append não deveria ficar nas tasks não? Já que eu tô cadastrando NA TASK???"
- "To com preguiça de fazer print, só isso — me vê as mensagens de erro, pf!"
- "Tu pode ir guardando nossos achados num relatório temporário à medida que progredimos?"
- Por que run listar diz "tarefa não encontrada" mesmo eu tendo cadastrado?

O que validei manualmente e como:
Testei cada comando na mão antes de aceitar qualquer trecho como pronto, registrado entrada por entrada no Diário de Tentativas. A validação mais importante foi contra os testes oficiais do professor via diff, não só teste manual — foi isso que revelou os bugs de isatty e buffering (decisão #6): eu sabia que a saída estava errada, mas não sabia por quê; a explicação técnica veio inteira da IA, e eu apliquei depois de entender o conceito (esse é o caso mais claro do projeto de solução técnica vinda inteiramente da IA).

8. Reflexão Final (5–8 linhas)

O que você melhoraria com mais tempo, principais decisões técnicas e erros que evitaria na próxima. Tudo junto, sem subseções separadas.

Reflexão:

Com mais duas horas eu trataria mais os erros, pensaria em mais casos que possam quebrar meu código e tentaria implementar o máximo possível de failsafes. A maior decisão técnica foi arquitetural: decidir quantos arquivos vão existir, quais ficam com quais funções e quais dependem dos outros. Originalmente nem tinha um common.h e eu pensei em fazer o job.c junto de task.c. Outra grande decisão foi sobre as funções que eu não sabia se iam pro task.c ou exec.c — no fim ficaram em exec.c. Não ter usado os testes do professor como referência me causou muito backtracking e me fez buscar soluções de emergência e mais rápidas. Dei preferência para testes de funções locais e acabei não dando a importância devida aos testes que exigem a saída específica.

9. Checklist Final de Entrega

Item	Confirmado?
Compilei do zero seguindo só o que está no relatório	Sim — git pull numa segunda máquina + make clean && make,

Item	Confirmado?
	gerou processflow limpo, Diário #26
Rodei os testes e evidencias.log foi gerado	Sim — log iniciado 22/08 09:22:47, contínuo até 24/08 (script -a), incluindo a recriação documentada dos bugs pra print (Seção 6)
Tenho prints obrigatórios	Sim (24/08) — validação final (3 diffs oficiais) e erro principal (recriação de isatty/buffering, com nota de transparência)
Testei pelo menos um caso limite e um caso inválido	Sim — Diário #13, #19, #25 (testes oficiais do professor)
Preenchi a seção de uso de IA	Sim
Revisei o relatório e removi frases genéricas / vazias	Sim — revisão final feita, sem clichês genéricos encontrados

10. Se eu tivesse mais 2 horas

Aplique as instruções abaixo de forma objetiva e verificável, evitando texto genérico.

Escreva 5-10 linhas sobre o que você melhoraria (robustez, testes, warnings, performance, organização).

- Reforçar mais o código: mais validação defensiva além do que já existe hoje (limites de buffer, retorno de chamadas de sistema checado em mais pontos).
- Deixar o código mais à prova de erro com mais testes — exemplos: validar tamanho de nome/input/output antes do strcpy (buffer overflow potencial), distinguir "terminou com sucesso" de "terminou com erro" via WEXITSTATUS (não só WIFEXITED), tratar o caso degenerado de run pipe com uma única task (n==1) e input/output/append no meio da cadeia, detectar processo morto de forma anormal em jobs/wait, checar task <nome> sem programa (argv[2]==NULL).
- Testes automáticos de verdade — script rodando os 3 testes oficiais e os próprios via diff, em vez da validação manual de hoje (tudo rodado à mão a cada mudança).
- Makefile mais profissional — compilação incremental via .o (não recompilar tudo a cada make), flags -Wall -Wextra, alvos .PHONY explícitos.

Regras de Integridade (não mudam)

- Ocorrência relevante sem evidência (log ou print) = não conta para a nota.
- Contradição entre prints, log e relato = relatório perde credibilidade.
- Indício de evidência fabricada = entrega invalidada.